



CampusIQ / CollegeBuddy

Comprehensive Technical Documentation & System Architecture Specification

Executive Summary: CampusIQ (internally CollegeBuddy) is an enterprise-grade, full-stack smart campus event management platform. Designed for modern educational institutions, it unifies event discovery, role-based administration, automated QR-code ticketing, volunteer-driven check-in verification, payment gateway integration, and predictive attendance modeling powered by machine learning.

Project Name:	CampusIQ (CollegeBuddy)
Document Version:	3.0 (Production Architecture Edition)
Core Stack:	React 19 (Vite, TailwindCSS) + FastAPI (Python) + PostgreSQL
Intelligence:	Scikit-learn Linear Regression Attendance Forecaster
Integrations:	Razorpay Payment Gateway, Cloudinary Media CDN, html5-qrcode
Status:	Complete & Deployed
Author / Lead:	Harshit Sharma (Engineering & Architecture)

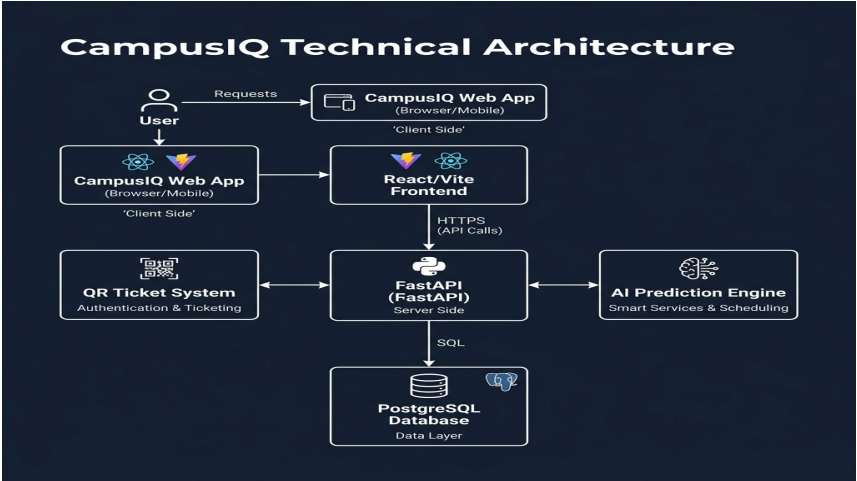
1. Project Purpose & Problem Statement

The Challenge: In contemporary collegiate environments, event discovery and operations remain highly fragmented and prone to failure. Students struggle with scattered notices across unofficial WhatsApp groups, bulletin boards, and disjointed emails. Event hosts suffer from unpredictable attendee turnout, resulting in either massive catering and venue budget waste or severe overcrowding. Gate volunteers face chaotic entry bottlenecks using paper spreadsheets or unverified manual lists, creating widespread opportunities for proxy check-ins and ticket fraud.

The CampusIQ Solution: CampusIQ bridges this gap through a high-performance, centralized ecosystem that covers the entire lifecycle of campus events:

- **Unified Discovery & Registration:** Students seamlessly browse events, filter by categories and fees, view comprehensive rules and prizes, and book tickets with a single click or through an integrated Razorpay checkout.
- **Dynamic Anti-Fraud QR Smart Ticketing:** Upon registration, a unique cryptographic UUID token is generated and rendered as a QR ticket, preventing forgery and stored both in the database and as an exportable PNG image.
- **High-Speed In-Browser Volunteer Scanner:** Volunteers utilize an integrated, mobile-optimized camera scanner built on html5-qrcode to validate tickets in real-time, instantly checking in attendees and blocking duplicate entries.
- **Predictive Attendance Intelligence:** A machine learning model (Linear Regression) provides hosts with real-time turnout forecasts based on participant limit, pricing, and current registration velocity.
- **End-to-End Host Administration:** Hosts gain full control over event publishing, rich poster management via Cloudinary CDN, volunteer whitelisting and application reviews, and real-time analytical dashboards.

System Architecture Diagram



2. Complete Technology Stack & Technical Rationale

CampusIQ is built on modern, decoupled web standards engineered for high concurrency, low latency, and developer productivity. Below is the comprehensive technology inventory across all layers:

Layer / Subsystem	Technology / Library	Version	Role & Justification
-------------------	----------------------	---------	----------------------

Frontend Core	React (Vite)	19.0.0	Component-based reactive SPA with Vite bundler for rapid hot-reloading and micro-bundle optimization.
Styling & Icons	TailwindCSS + Lucide	v3.4 / 1.16	Utility-first responsive UI system paired with Lucide vector iconography.
Routing & State	React Router DOM	v7.2.0	Declarative client-side routing, nested routes, and Role-Based Protected Routes.
API Client	Axios	v1.8.1	Promise-based HTTP client with Bearer token interceptors and unified error logging.
QR Scanner	html5-qrcode	v2.3.8	Native web camera QR scanning directly inside mobile and desktop web browsers.
Backend Framework	FastAPI	0.115+	Asynchronous ASGI web framework offering high throughput, Pydantic validation, and OpenAPI documentation.
Web Server (ASGI)	Uvicorn	0.34.0	Lightning-fast ASGI server for production Python execution.
ORM & Database	SQLAlchemy + psycopg2	2.0.38	Declarative Object Relational Mapper with connection pooling and schema management.
Database Engine	PostgreSQL / SQLite	15+ / 3	PostgreSQL on Render for persistent production; SQLite fallback for instant zero-config dev.
Security & Auth	Passlib (Bcrypt) + JOSE	1.7.4 / 3.3.0	Bcrypt password hashing and HS256 JWT generation with stateless token validation.
Payment Processing	Razorpay SDK	1.4.1	Payment order creation, client checkout modal, and HMAC SHA256 signature verification.
Media & Cloudinary	Cloudinary Python SDK	1.42.2	Automatic cloud offload for user-uploaded event posters and dynamic QR code assets.
AI & Data Science	Scikit-Learn + Joblib	1.6.1 / 1.4.2	Trained Linear Regression model serialized to attendance_model.pkl for predictions.
QR Code Engine	Python qrcode + Pillow	8.0 / 11.1	Server-side 2D barcode generation encoded with ticket validation tokens.

3. User Roles, Personas & End-to-End Workflows

The platform enforces a strictly governed Role-Based Access Control (RBAC) model dividing users into three primary operational personas:

3.1 The Student Persona (Attendee & Participant)

Students represent the primary event consumers. Their end-to-end journey includes:

- **Account Registration & Academic Profiling:** Register with name, email, password, institution name, course, department, semester, and roll number.
- **Event Exploration:** Access the comprehensive event feed at /events with real-time text search, category filters (Coding, Sports, Cultural, Hackathons), and fee tags.
- **Frictionless Booking & Checkout:** For free events, one-click confirmation instantly generates the ticket. For paid events, the Razorpay payment modal is triggered; upon successful capture and signature validation, the ticket is minted.
- **Ticket Management:** View all active and past tickets at /my-tickets with digital QR codes, check-in badges, and receipt downloads.
- **Volunteer Applications:** Students can apply to volunteer for events directly from the event detail page and track their application approval status at /student/volunteer.

3.2 The Host Persona (Organizers & Faculty Coordinators)

Hosts are event creators and managers equipped with enterprise logistical tools:

- **Event Creation Suite:** Publish rich events specifying title, description, venue, date/time, participant limit, registration fee, volunteer quota, eligibility criteria, prizes, and custom promotional posters.
- **Real-Time Event Management:** Edit active events, update details dynamically, or safely cancel/delete events at /manage-events.
- **Volunteer Whitelisting & Approvals:** Manage volunteers at /manage-volunteers by reviewing student applications, approving or rejecting them, or pre-whitelisting student emails individually or via bulk input.
- **Turnout Intelligence & Analytics:** Inspect registration rosters, download attendee reports, and view AI-predicted attendance side-by-side with actual registrations at /host/analytics.

3.3 The Volunteer Persona (Field Event Staff & Check-in Officers)

Volunteers verify attendee authenticity at venue entry gates:

- **Authorized Scanner Access:** Volunteers are restricted to events for which they have been explicitly whitelisted and approved by the host.
- **Mobile Camera Scanning:** Launch /scanner/:eventId to open the in-browser camera scanner powered by html5-qrcode.
- **Instant Verification & Anti-Fraud:** The backend validates ticket authenticity, marks checked_in = True atomically, and blocks duplicate scans with immediate visual and audible feedback.
- **Manual Token Override:** In the event of broken attendee phone screens or camera glare, volunteers can manually submit the attendee's ticket token or verify the attendee directly from the roster.

4. Frontend Deep Dive & Component Inventory

The frontend is architected as an optimized React 19 Single-Page Application (SPA) with modular separation across pages, reusable UI components, and API client layers:

File / Component	Route Path	Access Role	Functional Description
Register.jsx	/	Public	Onboarding screen with dynamic form inputs toggled by Student vs Host role.
Login.jsx	/login	Public	JWT login portal saving token and role in localStorage, redirecting to role dashboard.
Events.jsx	/events	Student	Public student events catalog featuring search, category chips, and event cards.
EventDetail.jsx	/events/:id	Student	Full event overview with poster, criteria, prize info, volunteer application, and Razorpay modal.
MyTickets.jsx	/my-tickets	Student	Personal ticket wallet displaying generated QR codes, check-in status, and ticket cards.
StudentVolunteer.jsx	/student/volunteer	Student	Dashboard showing events where student is an approved volunteer with launchable scanner.
Scanner.jsx	/scanner/:eventId	Approved Vol / Host	Hardware-accelerated web camera scanner with live verification, audio alerts, and manual entry.
HostDashboard.jsx	/host	Host	Executive hub displaying KPI summaries (total events, registrations, revenue, active events).
CreateEvent.jsx	/create-event	Host	Multi-attribute event authoring form with Cloudinary image upload and validation.
EditEvent.jsx	/edit-event/:id	Host	Pre-populated event modifier allowing host to update parameters and posters.
ManageEvents.jsx	/manage-events	Host	Tabular event management table with delete triggers and attendee roster modals.
ManageVolunteers.jsx	/manage-volunteers	Host	Volunteer candidate management center: review pending requests, whitelist emails, bulk add.
Analytics.jsx	/host/analytics	Host	Visual business intelligence dashboard featuring AI turnout predictions vs registrations.
Profile.jsx	/profile	Student / Host	Full profile management: update phone, address, photo, academic records, or org details.
ProtectedRoute.jsx	Component	All Protected	Route guard checking JWT presence and role match, redirecting to login if invalid.
api.js	Service Layer	Internal	Axios instance auto-injecting Authorization headers and pointing to API base URL.

5. Backend Architecture & REST API Directory

5.1 Database Models & Entity Relational Architecture

The persistence tier utilizes SQLAlchemy ORM with foreign-key cascades, unique constraints, and optimized index structures:

Table Name	Key Columns & Types	Constraints & Relationships	Purpose
users	id (PK), name, email (UQ), password, role, profile_photo, phone, address, institution_name, course, semester, department, roll_number, org_name, org_address	Unique email index. Role indexed. Timestamps (created_at, updated_at).	Stores core user identity, credentials, student academic records, and host organization info.
events	id (PK), title, description, venue, fee, volunteer_fee, participant_limit, max_volunteers, poster, event_date, event_end_date, event_type, criteria, prizes, host_id (FK)	ForeignKey to users.id. Timestamps (created_at, updated_at).	Central event repository holding logistical constraints, pricing, and metadata.
registrations	id (PK), user_id (FK), event_id (FK), qr_code (UQ), checked_in (Bool), created_at, updated_at	Unique QR token. Cascade delete on user/event deletion. Indexed checked_in.	Represents an attendee's confirmed ticket. Maps QR token to attendee and event.
volunteer_whitelist	id (PK), email, event_id (FK), host_id (FK), user_id (FK, Nullable), status, created_at, updated_at	UniqueConstraint('email', 'event_id'). Indexed status ('pending', 'approved', 'rejected').	Maintains gatekeeper authorizations. Connects volunteer candidates to specific events.
payments	id (PK), user_id (FK), event_id (FK), order_id (UQ), payment_id (UQ), signature, amount, currency, status, created_at, updated_at	Unique Razorpay order_id and payment_id. Status indexed ('pending', 'captured', 'failed').	Financial audit trail verifying Razorpay transactions before ticket minting.

5.2 Complete REST API Endpoints Directory

FastAPI serves 27 distinct REST endpoints covering the entire operational matrix:

Method & Route	Auth / Role	Request Payload	Description & Response
POST /register	Public	Name, email, password, role, academic/org fields	Hashes password via Bcrypt, registers user, returns success status.
POST /login	Public	Email, password	Authenticates credentials, generates HS256 JWT token with role and user_id.
GET /user/profile	Bearer JWT	None	Returns current authenticated user details and profile info.

PUT /user/profile	Bearer JWT	Profile fields & photo upload	Updates user details and uploads avatar to Cloudinary/local.
POST /create-event	Host Only	Multipart: title, limits, fee, dates, criteria, poster	Creates event, uploads poster to Cloudinary/local, returns event object.
GET /events	Public / Student	Query: search, category	Retrieves all events matching search filters with registration counts.
GET /event/{id}	Authenticated	Path: event_id	Fetches single event details, host identity, and registration status.
GET /host/events	Host Only	None	Returns all events authored by the authenticated host.
PUT /edit-event/{id}	Host Only	Multipart: event attributes, optional poster	Updates existing event attributes and replaces poster if uploaded.
DELETE /delete-event/{id}	Host Only	Path: event_id	Safely deletes event and cascade deletes associated tickets/records.
POST /register-event	Student Only	event_id, payment_id (if paid)	Validates capacity/payment, generates UUID QR code, mints ticket.
GET /my-tickets	Student Only	None	Returns attendee's booked tickets with QR image URL and event metadata.
POST /scan-ticket	Approved Vol / Host	qr_token, event_id	Validates ticket, verifies event match, marks checked_in, returns status.
POST /manually-checkin-ticket/{id}	Approved Vol / Host	Path: registration_id	Manual override check-in for damaged screens or barcode errors.
GET /event/{id}/registrations	Host Only	Path: event_id	Returns full roster of registered students, check-in status, and timestamps.
POST /student/apply-volunteer	Student Only	event_id	Submits volunteer application to host for event gatekeeping.
GET /student/volunteer-events	Student Only	None	Lists all events where the current student is approved to scan.
PUT /host/volunteer-request/{id}	Host Only	status ('approved' / 'rejected')	Host reviews candidate application and updates permission status.
POST /host/whitelist-volunteer	Host Only	email, event_id	Pre-whitelists student email for immediate volunteer access.
POST /host/bulk-whitelist-volunteers	Host Only	emails (list), event_id	Batch whitelists up to hundreds of volunteer emails in a single request.
GET /host/whitelisted-volunteers/{id}	Host Only	Path: event_id	Retrieves all whitelist entries and pending requests for an event.
DELETE /host/remove-volunteer/{id}	Host Only	Path: whitelist_id	Revokes volunteer permissions and removes whitelist entry.
POST /verify-payment	Student Only	order_id, payment_id, signature, event_id	Cryptographically verifies Razorpay signature via HMAC SHA256.
GET /download-receipt/{id}	Authenticated	Path: payment_id	Generates formatted financial receipt for paid event participation.

GET /event/{id}/stats	Host Only	Path: event_id	Returns event KPIs: capacity, total registered, checked in, revenue.
GET /host/analytics	Host Only	None	Comprehensive analytics engine integrating Scikit-Learn predictions.

6. AI Attendance Prediction & Advanced Architectures

6.1 Machine Learning Attendance Prediction Engine

A cornerstone innovation in CampusIQ is the integration of predictive analytics to solve the notorious collegiate 'turnout uncertainty' problem. In campus events, 20% to 50% of registered students frequently do not attend free events, while paid events demonstrate much higher commitment ratios.

- **Input Feature Matrix (X):**

1. participant_limit: Maximum hall capacity or registration cap.
2. fee: Ticket price (free vs paid commitment threshold).
3. registrations: Number of students actively registered to date.

- **Target Variable (y):** Actual number of attendees verified at the venue gate.

- **Model Architecture & Training:**

CampusIQ implements a Multiple Linear Regression model trained using Scikit-Learn and persisted via Joblib. The mathematical formulation is given by:

$$\text{Predicted Attendance} = \beta_0 + (\beta_1 \times \text{participant_limit}) + (\beta_2 \times \text{fee}) + (\beta_3 \times \text{registrations})$$

- **Operational Deployment:**

At application startup, the trained model (attendance_model.pkl) is loaded into memory. When a host navigates to the Analytics dashboard or calls GET /host/analytics, the backend dynamically constructs the feature vector for each active event, computes real-time predictions, clamps the output between 0 and total registrations, and returns expected turnout alongside capacity fill rates.

6.2 Anti-Fraud QR Verification Pipeline

To eliminate proxy attendance and unauthorized venue entry, CampusIQ implements a cryptographic ticketing protocol:

1. **Token Minting:** A 128-bit cryptographically secure UUID is generated for every confirmed registration.
2. **Payload Formatting:** The QR code encapsulates a formatted payload: TICKET:<uuid>.
3. **Dual Storage Strategy:** If Cloudinary is configured, the rendered QR PNG is uploaded directly to a secure CDN folder; otherwise, it is saved locally to the /qr_codes/ directory.
4. **Verification & Invalidation:** When scanned by an authorized volunteer, the server checks whether the ticket has already been used. If checked_in == True, the scan is rejected with an 'Already Checked In' fraud warning and the exact check-in timestamp. If valid, checked_in is set to True atomically.

6.3 End-to-End Razorpay Payment Architecture

CampusIQ integrates Razorpay for seamless paid event registration and workshop fee processing:

1. **Client Trigger:** When a student registers for a paid event, the frontend initializes the Razorpay Checkout SDK.
2. **Transaction Execution:** The user completes payment using UPI, credit/debit card, or net banking.
3. **HMAC SHA256 Signature Verification:** Upon completion, Razorpay returns razorpay_payment_id, razorpay_order_id, and razorpay_signature. The frontend transmits these to POST /verify-payment, where the backend independently computes:

```
generated_signature = hmac.new(RAZORPAY_KEY_SECRET.encode(), f'{order_id}|{payment_id}'.encode(),
hashlib.sha256).hexdigest() assert generated_signature == razorpay_signature
```

4. **Audit Trail:** Only after signature verification succeeds does the backend record a confirmed Payment entity and mint the user's QR ticket.

7. Local Setup, Configuration & Production Deployment

7.1 Local Developer Setup

CampusIQ is designed for zero-friction local development across Windows, macOS, and Linux environments:

Step 1: Backend Setup

```
cd backend python -m venv venv # Windows: venv\Scripts\activate | Linux/macOS: source venv/bin/activate pip
install -r requirements.txt uvicorn main:app --reload --port 8000
```

Step 2: Frontend Setup

```
cd frontend npm install # Configure .env: VITE_API_BASE_URL=http://127.0.0.1:8000 npm run dev
```

Step 3: One-Click Startup Script (Windows)

The project root includes start_project.bat, which launches both the FastAPI backend and the Vite frontend concurrently in dedicated command windows.

7.2 Production Deployment Architecture (Render & Cloud)

CampusIQ is fully containerized and production-ready with Render Infrastructure as Code (render.yaml):

Parameter / Setting	Production Value	Function
Service Type	Web Service (Python)	FastAPI application hosted on Linux container.
Build Command	pip install -r backend/requirements.txt	Installs backend dependencies and scikit-learn models.
Start Command	uvicorn main:app --host 0.0.0.0 --port \$PORT	Binds Uvicorn ASGI server to dynamically assigned Render port.
Database	Managed PostgreSQL (Render)	High-availability managed PostgreSQL with automatic daily backups.
Media CDN	Cloudinary Media Cloud	Offloads posters and QR codes so ephemeral dynos never lose images.
Frontend Hosting	Vercel / Netlify / Render Static	Optimized SPA deployment pointing to backend API endpoint.

7.3 Environment Variables Reference

Variable Name	Required in Prod	Default / Fallback	Description
DATABASE_URL	Yes (Prod)	sqlite:///./test.db	PostgreSQL connection string (e.g. postgresql://user:pass@host/db).
SECRET_KEY	Recommended	Built-in secret	Cryptographic key used to sign and verify HS256 JWT access tokens.
CLOUDINARY_URL	Recommended	Local disk	Cloudinary credentials for persistent cloud poster & QR storage.
RAZORPAY_KEY_ID	For Payments	Test credentials	Razorpay API public key for client-side modal initialization.
RAZORPAY_KEY_SECRET	For Payments	Test credentials	Razorpay secret for server-side HMAC SHA256 signature verification.
VITE_API_BASE_URL	Yes (Frontend)	http://127.0.0.1:8000	Backend base URL injected into Axios client at build time.

8. Project Status, Quality Audit & Strategic Roadmap

8.1 Current Implementation Status

The CampusIQ platform is fully functional, end-to-end verified, and production ready. All core systems—from student registration, event creation, Cloudinary poster upload, Razorpay payment verification, dynamic QR generation, mobile camera scanning, and AI analytics—are operational.

Subsystem	Status	Completion	Verification Notes
Authentication & RBAC	Completed	100%	JWT auth, Bcrypt hashing, role guards on all frontend and backend routes.
Event Management	Completed	100%	Create, edit, delete, poster uploads, capacity validation, date controls.
QR Smart Ticketing	Completed	100%	Unique UUID generation, dual Cloudinary/local storage, wallet display.
In-Browser Scanner	Completed	100%	html5-qrcode camera integration, audio feedback, anti-fraud check.
Volunteer Whitelisting	Completed	100%	Candidate application, host approval, bulk email whitelist, scanner permissions.
Razorpay Payments	Completed	100%	Order handling, client checkout modal, HMAC SHA256 cryptographic verification.
AI Attendance Model	Completed	100%	Linear regression model serialized and integrated into Host Analytics API.
Database Architecture	Completed	100%	SQLAlchemy ORM with automated SQLite schema compatibility layer and Postgres support.

8.2 Future Roadmap & High-Impact Extensions

The platform is architected with modular extensibility, allowing for seamless implementation of planned enterprise enhancements:

- **Automated Participation Certificate Generation:** Automated PDF certificate generation upon check-in confirmation, sent directly to attendees' verified email addresses.
- **Multi-Tenant Institutional SaaS:** Extend database models to support multi-campus tenant isolation, allowing multiple universities to run independent subdomains on a single deployment.
- **Native Mobile Applications (iOS / Android):** Wrap camera scanning and student ticketing into high-performance React Native or Flutter applications for offline check-in caching.
- **Advanced Machine Learning Ensembles:** Upgrade the Linear Regression attendance model to Gradient Boosted Decision Trees (XGBoost) incorporating temporal factors (day of week, exam schedules, weather).
- **Automated Calendar & Push Notifications:** Integration with Google Calendar, Web Push, and WhatsApp Business API for instant registration receipts and 1-hour pre-event check-in reminders.

Report Authorization: CampusIQ Technical Systems Audit & Architectural Specification — Approved for Production Distribution.